Software Architecture

Architectural Styles and Patterns

Saqif Kaisar

Department of CSE, UIU

CSE 3421 – Software Engineering

Why This Lesson?

Main Question

How should a software system be divided and organized so that developers can build, understand, deploy, and change it effectively?

Why Software Architecture Matters

It is essential to learn how software is **modularized** and **distributed into manageable pieces**.

Simple Example – University Portal

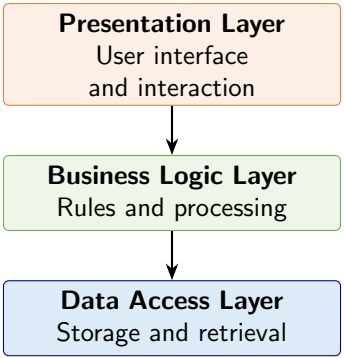
Instead of placing login, course registration, payment, and result processing in one large block, architecture defines clear components and how those components communicate.

N-Tier (Layered) Architecture

Main Idea

Organize a system into layers or tiers, each with a distinct responsibility.

Basic Layered Structure



Simple Example

A login page collects credentials, the business layer checks the login rules, and the data-access layer reads the user record from storage.

What Is N-Tier Architecture?

Definition

An approach where a project or program is distributed among **N separate computers** in a distributed network.

- A **layer** describes a logical responsibility.
- A **tier** describes a physical deployment location or machine.
- Layers may be placed on separate tiers to improve distribution and control.

Simple Example

A browser runs on a user's computer, application logic runs on a web server, and data is stored on a separate database server.

Types of N-Tier Architecture

3-Tier

Client, application server, and database server are separate.

2-Tier

Client communicates directly with a database server.

1-Tier

Interface, logic, and data are on one machine or application.

More tiers generally provide stronger separation, but also add deployment and communication complexity.

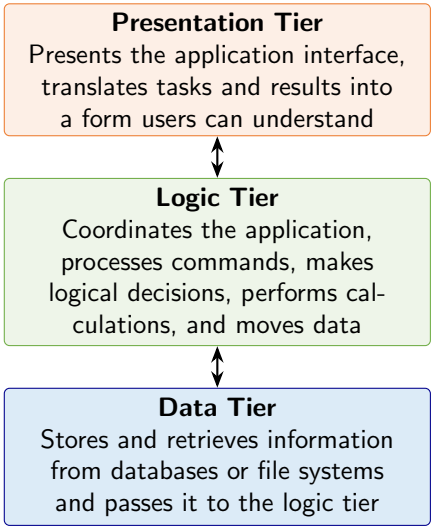
3-Tier Architecture



Simple Example – Online Shopping

The browser displays products, the application server calculates totals and validates orders, and the database stores products and transactions.

3-Tier Architecture – Detailed View



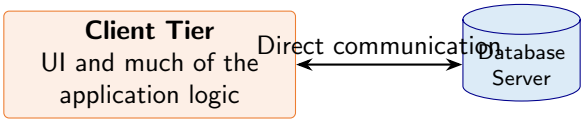
Typical Flow

1. User sends a request.
2. Logic processes it.
3. Data is queried or updated.
4. The result returns to the user.

Example

"Show all students with the last name Lee."

2-Tier Architecture

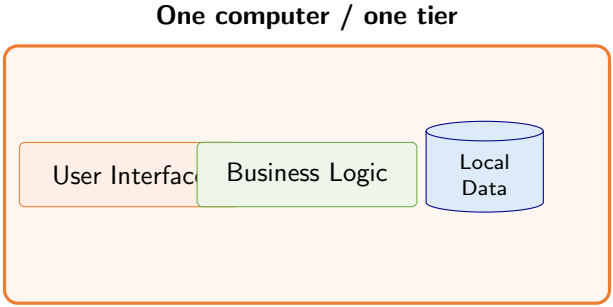


- The client sends queries directly to the database server.
- Often called a client–server architecture.

Simple Example

A desktop library application connects directly to a central database to search books and update borrowing records.

1-Tier Architecture



Simple Example

A standalone personal expense application stores its interface, calculation logic, and local data on the same computer.

Impacts of Multi-Tiering

Operational Qualities

- **Scalability:** tiers can be expanded independently.
- **Reliability:** responsibilities are isolated.
- **Fault tolerance:** failures can be contained or recovered.
- **Security:** direct access to sensitive tiers can be restricted.

Development Qualities

- **Maintainability:** changes stay within focused tiers.
- **Reusability:** shared logic or services can support many clients.

Example

Add more application servers during registration week without changing the database or student browsers.

Software Architectural Patterns

Main Idea

Reusable high-level structures for organizing components and their interactions.

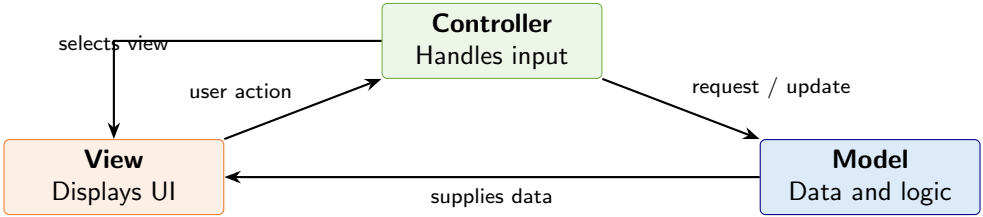
Types of Layered Architectural Pattern

- **MVC** – Model–View–Controller
- **HMVC** – Hierarchical MVC
- **MVVM** – Model–View–ViewModel
- **MVP** – Model–View–Presenter

Common Goal

Separate user-interface concerns from data and application logic so that each part can evolve with less impact on the others.

MVC



Simple Example – Student Result Page

The student clicks "View Result," the controller handles the request, the model retrieves grades, and the view displays the result table.

MVC Components

Model

Contains the data-related logic that the user works with. It represents data transferred between the View and Controller components.

View

Contains the UI logic of the application. A Customer view may include text boxes, dropdowns, buttons, tables, and other visible controls.

Controller

Acts as an interface between Model and View: processes incoming requests and business logic, manipulates data through the Model, and selects Views to render the final output.

MVC Frameworks

- **Spring** – Java
- **Django** – Python
- **Rails** – Ruby
- **Laravel** – PHP

Simple Example

In Django, a URL directs a request to a view function, models access data, and a template produces the page shown to the user.

Advantages of MVC

- **Rapid development:** developers can work on separate concerns.
- **Low coupling:** interface and data logic depend less directly on each other.
- **Efficient debugging:** defects can be located within a specific component.
- **High maintainability:** changes are easier to isolate and manage.

Simple Example

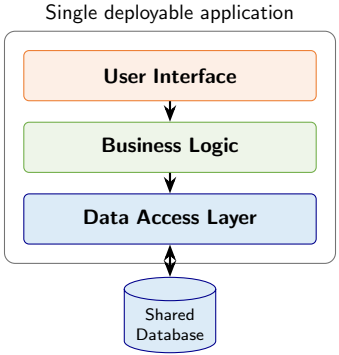
A designer can update the result-page layout without rewriting the grade calculation logic in the Model.

Monolithic Architecture

Main Idea

Build the complete application as one deployable unit.

Monolithic Architecture – Structure



Simple Example

A university portal packages login, registration, payment, and results into one application deployed together.

Monolithic Architecture – Characteristics

- Built as a **single and indivisible unit**; all functions are managed and served in one place.
- **Low-cost architecture** for small or early systems.
- Simple for development and deployment.
- Difficult for maintenance and scalability as the system grows.
- Often creates **tight coupling** between features.

Simple Example

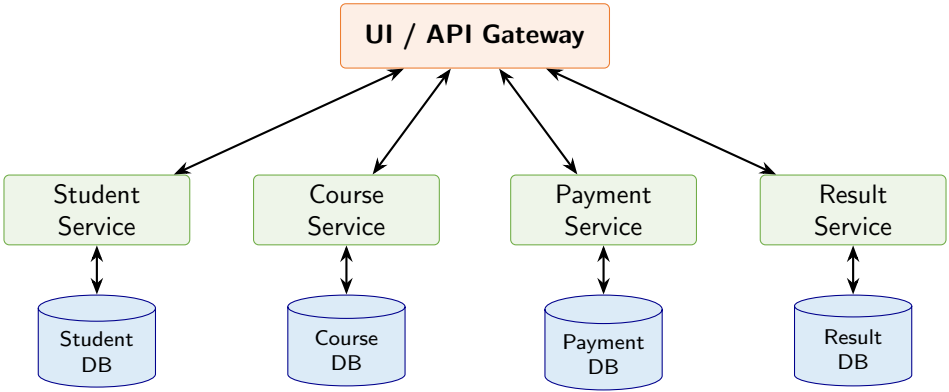
A small change to the library-fine feature may require testing and redeploying the entire university portal.

Microservice Architecture

Main Idea

Divide the application into small, independently developed and deployed services.

Microservice Architecture – Structure



Microservice Architecture – Characteristics

- Breaks the application into a collection of smaller independent units.
- Each service has its own logic and database and performs specific functions.
- Services communicate using lightweight mechanisms, often an HTTP resource API.
- Provides easy maintainability, scalability, and low coupling.
- Adds extra complexity, cross-cutting concerns, and cost.

Simple Example

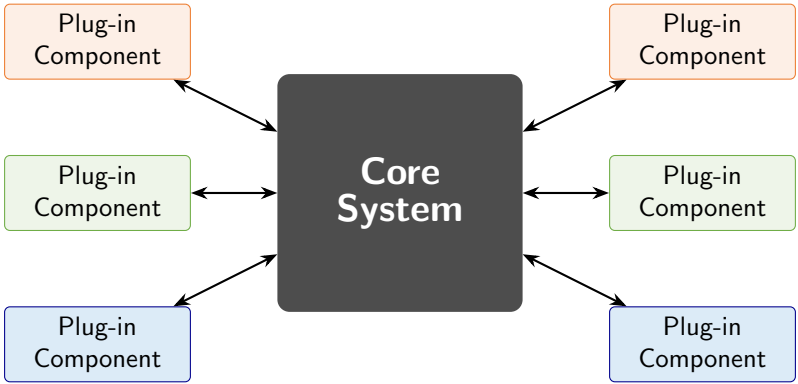
During fee-payment week, scale only the Payment Service. However, logging, authentication, monitoring, and error handling must work across all services.

Micro-Kernel Architecture Pattern

Also Known As

Plug-in architecture.

Micro-Kernel Architecture – Structure



Micro-Kernel Architecture – Characteristics

- Also called plug-in architecture; consists of a core system and several plug-in modules.
- The core provides minimal functionality needed to keep the system operational.
- Plug-in modules are independent components with specialized processing.
- Suitable when core routines are fixed but rules or features change frequently.
- Plug-ins need good **handshaking code** so that the microkernel detects installation and is ready to work with them.

Simple Example

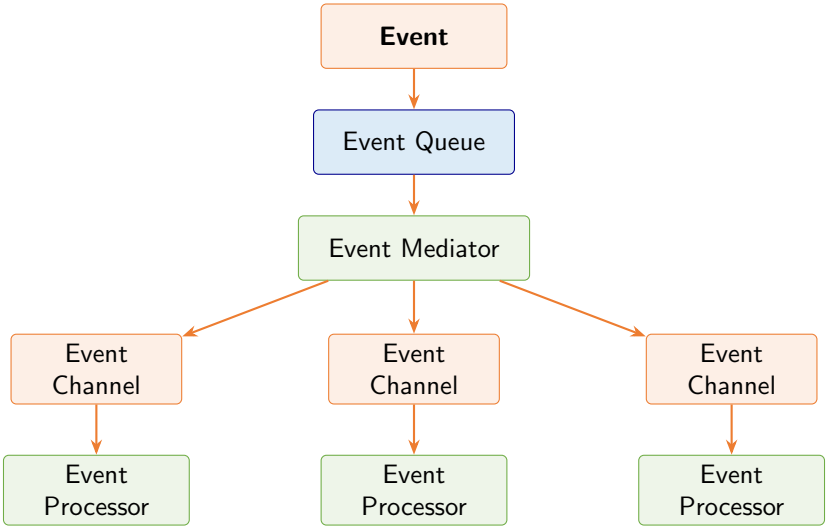
An IDE provides editing and file management in its core, while plug-ins add Java support, Git integration, themes, or database tools.

Event-Driven Architecture

Main Idea

Components communicate by producing, routing, and reacting to events.

Event-Driven Architecture – Structure



Event-Driven Architecture – Characteristics

- Made of decoupled, single-purpose event-processing components that asynchronously receive and process events.
- Agile and highly performant.
- Supports easy debugging, fast development, and low coupling.

Simple Example – Online Order

An `OrderPlaced` event can independently trigger payment processing, inventory updates, email confirmation, and delivery scheduling. The order component does not call each receiver directly.

Thank You

Questions?